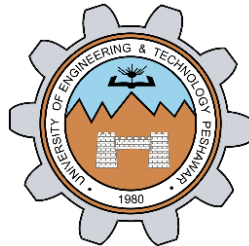


Lab # 8

Queues



Spring 2026

Data Structures And Algorithms Lab

Submitted by: **M Mohsin Khan**

Registration No.: **24PWCSE2382**

Class Section: **C**

Submitted to:

Engr. Abdullah Hamid

June 21 , 2026

Department of Computer Systems Engineering
University of Engineering and Technology, Peshawar

Task 01: Queues Using Arrays:

```
#include <iostream>
using namespace std;

#define MAX 100

struct Queue
{
    int arr[MAX];
    int front;
    int rear;
    int count;
};

void createQueue(Queue *q)
{
    q->front = 0;
    q->rear = -1;
    q->count = 0;
}

bool isEmpty(Queue *q)
{
    return q->count == 0;
}

bool isFull(Queue *q)
{
    return q->count == MAX;
}

void enqueue(Queue *q, int value)
{
    if (isFull(q))
    {
        cout << "Queue Overflow\n";
        return;
    }

    q->rear = (q->rear + 1) % MAX;
    q->arr[q->rear] = value;
    q->count++;
}

void dequeue(Queue *q)
{
    if (isEmpty(q))
    {
        cout << "Queue Underflow\n";
```

```
        return;
    }

    q->front = (q->front + 1) % MAX;
    q->count--;
}

int frontElement(Queue *q)
{
    if (isEmpty(q))
    {
        cout << "Queue is Empty\n";
        return -1;
    }

    return q->arr[q->front];
}

int rearElement(Queue *q)
{
    if (isEmpty(q))
    {
        cout << "Queue is Empty\n";
        return -1;
    }

    return q->arr[q->rear];
}

int size(Queue *q)
{
    return q->count;
}

int main()
{
    Queue q;
    createQueue(&q);

    cout << "Queue Created\n\n";

    // Enqueue elements
    enqueue(&q, 10);
    enqueue(&q, 20);
    enqueue(&q, 30);
    enqueue(&q, 40);
    enqueue(&q, 50);
```

```

        cout << "After Enqueue Operations:\n";
        cout << "Front Element: " <<
frontElement(&q) << endl;
        cout << "Rear Element: " <<
rearElement(&q) << endl;
        cout << "Queue Size: " << size(&q) <<
endl;
        cout << "Is Queue Empty? "
<< (isEmpty(&q) ? "Yes" : "No") <<
endl;
        cout << "Is Queue Full? "
<< (isFull(&q) ? "Yes" : "No") <<
endl;

        cout << "\nDequeuing Two
Elements...\n";
        dequeue(&q);
        dequeue(&q);

        cout << "Front Element: " <<
frontElement(&q) << endl;
        cout << "Rear Element: " <<
rearElement(&q) << endl;
        cout << "Queue Size: " << size(&q) <<
endl;

        cout << "\nAdding More Elements...\n";
        enqueue(&q, 60);
        enqueue(&q, 70);

```

```

        cout << "Front Element: " <<
frontElement(&q) << endl;
        cout << "Rear Element: " <<
rearElement(&q) << endl;
        cout << "Queue Size: " << size(&q) <<
endl;

        cout << "\nRemoving All Elements...\n";
        while (!isEmpty(&q))
        {
            cout << "Removed: " <<
frontElement(&q) << endl;
            dequeue(&q);
        }

        cout << "\nQueue Size After Emptying: "
<< size(&q) << endl;
        cout << "Is Queue Empty? "
<< (isEmpty(&q) ? "Yes" : "No") <<
endl;

        // Underflow test
        cout << "\nAttempting Dequeue on Empty
Queue:\n";
        dequeue(&q);

        return 0;
}

```

Terminal Output:

```

PS D:\4th Semester\DSA\lab_08_Queue> .\a.exe
Queue Created Successfully

After Enqueue Operations:
Front Element: 10
Rear Element: 50
Queue Size: 5
Is Queue Empty? No
Is Queue Full? No

Dequeuing Two Elements...
Front Element: 30
Rear Element: 50
Queue Size: 3

Adding More Elements...
Front Element: 30
Rear Element: 70
Queue Size: 5

Removing All Elements...
Removed: 30
Removed: 40
Removed: 50
Removed: 60
Removed: 70

Queue Size After Emptying: 0
Is Queue Empty? Yes

Attempting Dequeue on Empty Queue:
Queue Underflow

```

Task 2: Queue using Linked List

```
#include <iostream>

#include <cstdlib>
using namespace std;
// Node structure
struct Node
{
    int data;
    Node *next;
};
// Queue structure
struct Queue
{
    Node *front;
    Node *rear;
    int count;
};
// a) Creation
void createQueue(Queue *q)
{
    q->front = nullptr;
    q->rear = nullptr;
    q->count = 0;
}
// b) Enqueue
void enqueue(Queue *q, int value)
{
    Node *newNode = new Node;
    if (!newNode)
    {
        cout << "Memory allocation
failed\n";
        return;
    }
    newNode->data = value;
    newNode->next = nullptr;
    if (q->rear == nullptr)
    {
        // Empty queue
        q->front = q->rear = newNode;
    }
    else
    {
        q->rear->next = newNode;
        q->rear = newNode;
    }
    q->count++;
}
// c) Dequeue
```

```
void dequeue(Queue *q)
{
    if (q->front == nullptr)
    {
        cout << "Queue Underflow\n";
        return;
    }
    Node *temp = q->front;
    q->front = q->front->next;
    if (q->front == nullptr)
        q->rear = nullptr;
    delete temp;
    q->count--;
}
// d) Front element
int frontElement(Queue *q)
{
    if (q->front == nullptr)
    {
        cout << "Queue is Empty\n";
        return -1;
    }
    return q->front->data;
}
// e) Rear element
int rearElement(Queue *q)
{
    if (q->rear == nullptr)
    {
        cout << "Queue is Empty\n";
        return -1;
    }
    return q->rear->data;
}
// f) Size
int size(Queue *q)
{
    return q->count;
}
// g) Empty
bool isEmpty(Queue *q)
{
    return (q->front == nullptr);
}

int main()
{
```

```

Queue q;
createQueue(&q);

cout << "Queue Created
Successfully\n\n";

enqueue(&q, 10);
enqueue(&q, 20);
enqueue(&q, 30);
enqueue(&q, 40);
enqueue(&q, 50);

cout << "After Enqueue Operations:\n";
cout << "Front Element: " <<
frontElement(&q) << endl;
cout << "Rear Element: " <<
rearElement(&q) << endl;
cout << "Queue Size: " << size(&q) <<
endl;
cout << "Is Queue Empty? "
<< (isEmpty(&q) ? "Yes" : "No") <<
endl;

cout << "\nDequeuing Two
Elements...\n";
dequeue(&q);
dequeue(&q);

cout << "Front Element: " <<
frontElement(&q) << endl;
cout << "Rear Element: " <<
rearElement(&q) << endl;
cout << "Queue Size: " << size(&q) <<
endl;

```

```

cout << "\nAdding More Elements...\n";
enqueue(&q, 60);
enqueue(&q, 70);

cout << "Front Element: " <<
frontElement(&q) << endl;
cout << "Rear Element: " <<
rearElement(&q) << endl;
cout << "Queue Size: " << size(&q) <<
endl;

cout << "\nRemoving All Elements:\n";
while (!isEmpty(&q))
{
    cout << "Dequeued: "
        << frontElement(&q) << endl;
    dequeue(&q);
}

cout << "\nQueue Size: " << size(&q) <<
endl;
cout << "Is Queue Empty? "
<< (isEmpty(&q) ? "Yes" : "No") <<
endl;

// Underflow Test
cout << "\nAttempting Dequeue on Empty
Queue:\n";
dequeue(&q);

return 0;
}

```

Terminal:

```

PS D:\4th Semester\DSA\lab_08_Queues> .\a.exe
Queue Created Successfully

After Enqueue Operations:
Front Element: 10
Rear Element: 50
Queue Size: 5
Is Queue Empty? No

Dequeuing Two Elements...
Front Element: 30
Rear Element: 50
Queue Size: 3

Adding More Elements...
Front Element: 30
Rear Element: 70
Queue Size: 5

```

```

Adding More Elements...
Front Element: 30
Rear Element: 70
Queue Size: 5

Removing All Elements:
Dequeued: 30
Dequeued: 40
Dequeued: 50
Dequeued: 60
Dequeued: 70

Queue Size: 0
Is Queue Empty? Yes

Attempting Dequeue on Empty Queue:
Queue Underflow

```

RUBRICS (LABS/ PROJECTS):

Criteria & Point Assigned	Outstanding 4	Acceptable 3	Considerable 2	Below Expectations 1	Score
Valuing related to work, including responsibility, and professionalism.	Demonstrates excellent responsibility and professionalism in task management. Shows strong commitment to quality work, meeting the deadline, and comprehensive, well-organized effective documentation following all guidelines with exemplary attention to detail.	Demonstrates some responsibility and professionalism. Displays moderate commitment to quality work, meets the deadline and shows sufficient thoroughness in documentation, adhering to guidelines, with some room for enhancement.	Demonstrates limited effort in responsibility and professionalism. Misses the deadline with notable inconsistency in quality work and shows insufficient thoroughness or clarity in documentation, necessitating improvements.	Demonstrates a lack of responsibility and professionalism. Misses the deadline and lacks quality work. Documentation is incomplete, disorganized, or unclear.	
Ability to apply techniques, methods, and procedures effectively.	Executes lab tasks with exceptional skill to design/develop solution and shows proficiency in lab techniques and procedures.	Executes lab tasks with adequate skill to design/develop solution and shows proficiency in lab techniques and procedures.	Executes lab tasks with minimal skill to design/develop solution and shows moderate proficiency in lab techniques and procedures.	Executes lab tasks with limited skill to design/develop solution and lacks proficiency in lab techniques and procedures.	

Demonstrate understanding and relevant Knowledge.	Demonstrates exceptional understanding of concepts with clear explanation of investigation and analysis of the objectives and outcomes.	Demonstrates a good understanding of concepts with some explanations of investigation and analysis that align with the lab's objectives and outcomes.	Demonstrates a basic understanding of key concepts but lacks clarity or depth in explaining the investigation and analysis, with some inconsistencies.	Demonstrates limited understanding of concepts with unclear explanations of investigation and analysis that do not align with the lab's objectives and outcomes.
---	--	--	---	---